

Java Spring Boot Backend Development

Real-World Portfolio Backend Project

Secure, scalable and maintainable backend solutions built with Java and Spring Boot.

■ Java

■ REST API

■ JWT

■ Spring Boot

■ Spring Security

■ PostgreSQL

This case study is generated from the actual **portfolio-Backend** source code in this repository. All technical claims, APIs, entities, and security details map to implemented code.

Project Overview

The Personal Portfolio Backend powers portfolio content delivery and admin-side management through REST APIs. It separates public read endpoints (portfolio, skills, projects, education, experience, translations, settings) from protected write operations for content updates.

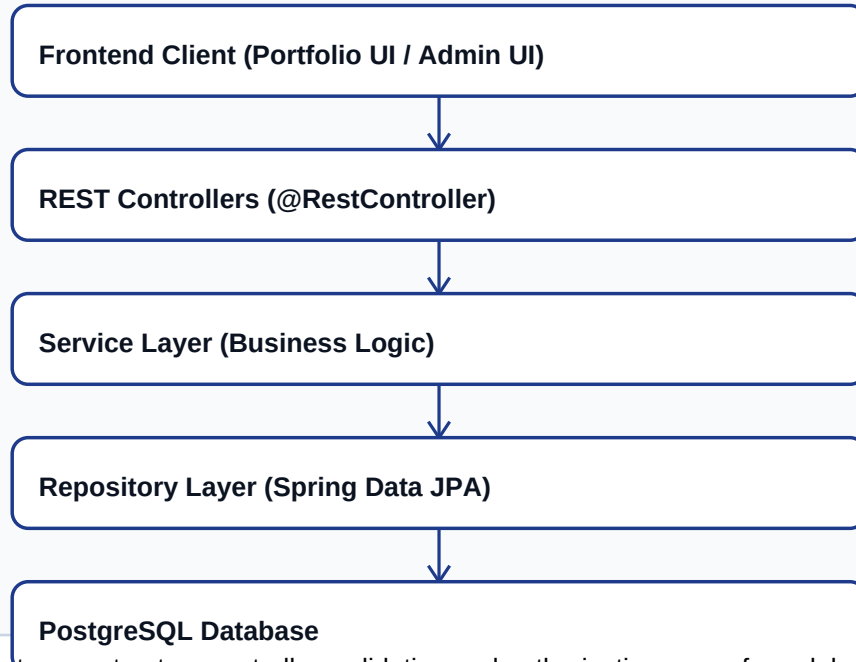
Implemented Functionality	What the backend currently provides
REST API development	Dedicated controllers for auth, users, profile, skills, projects, experience, education, contact, settings, etc.
Portfolio content management	Admin CRUD workflows for core portfolio sections through secured endpoints.
Database integration	Spring Data JPA entities and repositories with PostgreSQL runtime configuration.
CRUD operations	Create, read, update, delete operations across portfolio resources.
Authentication / authorization	JWT-based authentication with role-based restrictions in Spring Security.
Security	BCrypt password encoding, stateless sessions, JWT filter and custom auth error handling.
API communication	JSON request/response model with DTO validation and structured API responses.

Project runtime screenshot placeholder

Insert actual Spring Boot run screenshot (application startup/log + healthy API call) provided by project owner.

Backend Architecture

Request Processing Architecture



Request lifecycle: client request enters controller, validation and authorization are enforced, business logic executes in services, repositories persist/fetch entities, and JSON response is returned. JWT filter executes before protected controller methods.

Technology Stack

Area	Technologies used in the current backend
Backend	Java 21, Spring Boot 3.3.5, Spring MVC, Spring Data JPA, Hibernate
Security	Spring Security, JWT (jjwt), BCrypt password encoding, role-based access control
Database	PostgreSQL (primary runtime), MySQL connector present, H2 for development/testing support
API	REST APIs, JSON payloads, DTO validation
Development	Maven, Git/GitHub, IntelliJ IDEA-ready multi-package project structure

Technology markers: ■ Java | ■ Spring | ■ Security | ■ PostgreSQL

Project Structure

Core packages from `com.mohiudding.portfolio_Backend`:

controller	REST endpoint layer (auth, skills, projects, profile, contact, media, resume, translations, etc.)
service / service.impl	Business rules and orchestration for CRUD and auth flows
repository	JPA repositories for database interaction
model	Entity models (User, Profile, Skill, Project, Experience, Education, etc.)
dto	Request/response data contracts and validation
security	JWT service/filter + auth failure handlers
config	Security, CORS, web config, data initialization

IntelliJ project structure screenshot placeholder

Insert actual IntelliJ project tree screenshot showing controller/service/repository/model/dto/security/config packages.

Database Design (Entity View)



Key relation implemented in code: **Profile (profiles)** has **OneToMany** stats in **profile_stats**. Project technologies are stored via element collection table **project_technologies**. Other entities are independent CRUD tables managed through dedicated repositories.

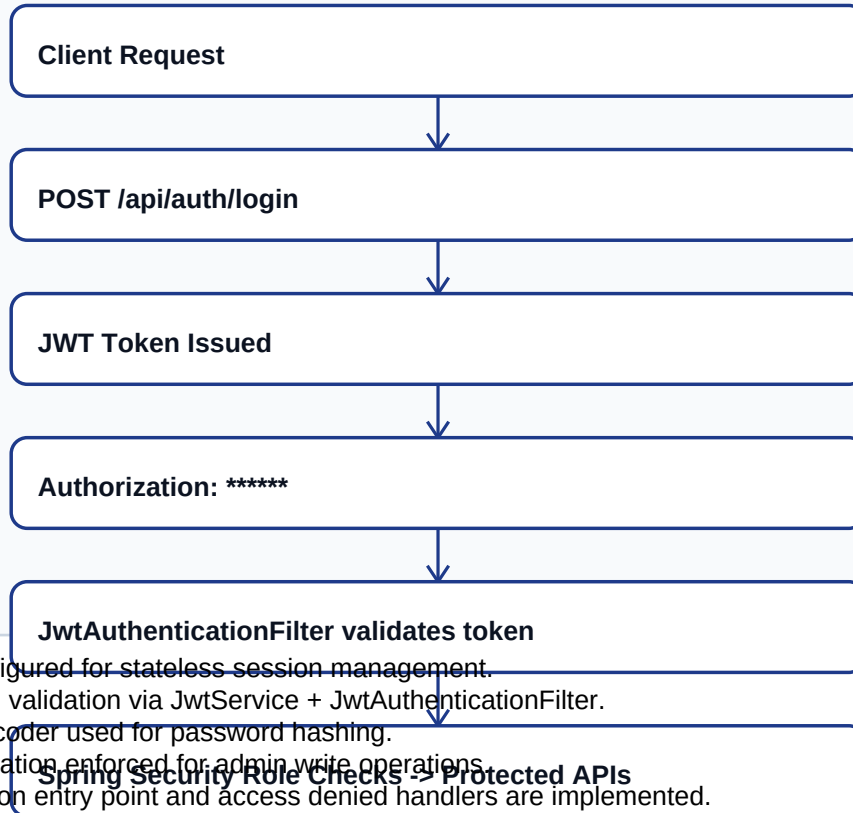
REST API Demonstration

Method	Endpoint	Purpose
GET	/api/skills	Retrieve all skills
POST	/api/skills	Create a new skill (ADMIN)
PUT	/api/skills/{id}	Update skill by ID (ADMIN)
DELETE	/api/skills/{id}	Delete skill by ID (ADMIN)
POST	/api/auth/login	Authenticate and return JWT token
GET	/api/projects/featured	Read featured projects
PATCH	/api/contact/messages/{id}/read	Mark contact message read/unread (ADMIN)

Postman / Swagger screenshot placeholder
Insert real endpoint request/response screenshots from your running backend (no mock data).

Security Implementation

Authentication & Authorization Flow



Database / API / Code Screenshots

Add your real high-resolution screenshots to finalize this page for Fiverr upload:

1) PostgreSQL tables view

Show actual entities/tables in DB client

2) Postman API response

Successful real endpoint response

3) IntelliJ backend code

Controller/service or security code snippet

4) Spring Boot running

Terminal/log startup + health endpoint

What I Can Deliver

- Java Spring Boot backend development
- REST API development
- CRUD operations
- PostgreSQL/MySQL integration
- Spring Security
- JWT authentication
- Role-based authorization
- API integration
- Backend bug fixing
- Existing Spring Boot project improvements
- Clean and maintainable backend architecture

Need a reliable Java Spring Boot backend for your project? Let's build it.